

```
"""
```

```
Image Captioning (CNN Encoder  
+ RNN Decoder)
```

```
-----  
-----
```

```
Combines computer vision and  
NLP: a pretrained ResNet-50  
extracts a  
feature vector from an image,  
and an LSTM decoder generates  
a caption  
word-by-word, conditioned on  
those features.
```

```
Requirements:
```

```
    pip install torch  
torchvision pillow
```

```
Dataset format expected (e.g.  
Flickr8k):
```

```
    images/
```

```
<- folder of .jpg images
    captions.txt
<- lines of
"image_filename,caption text"
```

Usage:

```
# 1. Train
python
image_captioning.py train --
images ./images --captions
./captions.txt --epochs 20

# 2. Generate a caption
for a new image
python
image_captioning.py caption -
-image ./test.jpg --
checkpoint ./caption_model.pt
"""
```

```
import argparse
import os
import csv
```

```
from collections import
Counter

import torch
import torch.nn as nn
from torch.utils.data import
Dataset, DataLoader
from torch.nn.utils.rnn
import pad_sequence
from torchvision import
models, transforms
from PIL import Image

# -----
-----
-----
# Vocabulary: maps words <->
integer ids
# -----
-----
-----
class Vocabulary:
```

```
PAD, START, END, UNK = "  
<pad>", "<start>", "<end>", "  
<unk>"
```

```
def __init__(self,  
freq_threshold=2):  
    self.freq_threshold =  
freq_threshold  
    self.word2idx =  
{self.PAD: 0, self.START: 1,  
self.END: 2, self.UNK: 3}  
    self.idx2word = {i: w  
for w, i in  
self.word2idx.items()}
```

```
def __len__(self):  
    return  
len(self.word2idx)
```

```
@staticmethod  
def tokenize(text):  
    return  
text.lower().strip().replace(
```

```
".", "").replace(",",",  
").split()  
  
    def build(self,  
sentence_list):  
        counter = Counter()  
        for sentence in  
sentence_list:  
  
counter.update(self.tokenize(  
sentence))  
  
        idx =  
len(self.word2idx)  
        for word, freq in  
counter.items():  
            if freq >=  
self.freq_threshold and word  
not in self.word2idx:  
  
self.word2idx[word] = idx  
  
self.idx2word[idx] = word
```

```

        idx += 1

    def numericalize(self,
text):
        tokens =
self.tokenize(text)
        return
[self.word2idx.get(t,
self.word2idx[self.UNK]) for
t in tokens]

# -----
-----
-----

# Dataset
# -----
-----
-----

class
CaptionDataset(Dataset):
    def __init__(self,
images_dir, captions_file,

```

```
vocab, transform):
    self.images_dir =
images_dir
    self.transform =
transform
    self.vocab = vocab
    self.samples = [] #
list of (filename, caption)

    with
open(captions_file,
newline="", encoding="utf-8")
as f:
        reader =
csv.reader(f)
        for row in
reader:
            if len(row) <
2:
                continue
            filename,
caption = row[0].strip(),
row[1].strip()
```

```
self.samples.append((filename  
, caption))
```

```
    def __len__(self):  
        return  
len(self.samples)
```

```
    def __getitem__(self,  
index):  
        filename, caption =  
self.samples[index]  
        image =  
Image.open(os.path.join(self.  
images_dir,  
filename)).convert("RGB")  
        image =  
self.transform(image)
```

```
        tokens =  
[self.vocab.word2idx[Vocabula  
ry.START]]  
        tokens +=
```

```

self.vocab.numericalize(caption)

        tokens +=
[self.vocab.word2idx[Vocabulary.
ry.END]]

        return image,
torch.tensor(tokens)

def collate_fn(batch):
    images, captions =
zip(*batch)
    images =
torch.stack(images, dim=0)
    captions =
pad_sequence(captions,
batch_first=True,
padding_value=0)
    return images, captions

IMAGE_TRANSFORM =
transforms.Compose([

```

```

        transforms.Resize((224,
224)),
        transforms.ToTensor(),

transforms.Normalize(mean=
[0.485, 0.456, 0.406],
                                std=
[0.229, 0.224, 0.225]),
])

# -----
-----
-----

# Model: ResNet-50 encoder
(frozen, pretrained) + LSTM
decoder
# -----
-----
-----

class EncoderCNN(nn.Module):
    def __init__(self,
embed_size):

```

```

        super().__init__()
        resnet =
models.resnet50(weights=models.
ResNet50_Weights.IMAGENET1K
_V2)
        for param in
resnet.parameters():

param.requires_grad = False
# freeze pretrained weights
        modules =
list(resnet.children())[:-1]
# drop final fc layer
        self.resnet =
nn.Sequential(*modules)
        self.fc =
nn.Linear(resnet.fc.in_featur
es, embed_size)

        def forward(self,
images):
            with torch.no_grad():
                features =

```

```
self.resnet(images)
# (batch, 2048, 1, 1)
    features =
features.reshape(features.size(0), -1)
    return
self.fc(features)
    # (batch, embed_size)
```

```
class DecoderRNN(nn.Module):
    def __init__(self,
embed_size, hidden_size,
vocab_size, num_layers=1):
        super().__init__()
        self.embed =
nn.Embedding(vocab_size,
embed_size)
        self.lstm =
nn.LSTM(embed_size,
hidden_size, num_layers,
batch_first=True)
        self.fc =
```

```

nn.Linear(hidden_size,
vocab_size)

    def forward(self,
features, captions):
        embeddings =
self.embed(captions[:,
:-1])          # skip
<end>
        inputs =
torch.cat((features.unsqueeze
(1), embeddings), dim=1)
        outputs, _ =
self.lstm(inputs)
        return
self.fc(outputs)

    def generate(self,
features, vocab, max_len=20):
        result_ids = []
        inputs =
features.unsqueeze(1)
        states = None

```

```

        for _ in
range(max_len):
            hiddens, states =
self.lstm(inputs, states)
            output =
self.fc(hiddens.squeeze(1))
            predicted =
output.argmax(1)

result_ids.append(predicted.i
tem())

            if
predicted.item() ==
vocab.word2idx[Vocabulary.END
]:
                break
            inputs =
self.embed(predicted).unsquee
ze(1)

        return result_ids

class

```

```
ImageCaptioningModel(nn.Module):
    def __init__(self,
embed_size, hidden_size,
vocab_size, num_layers=1):
        super().__init__()
        self.encoder =
EncoderCNN(embed_size)
        self.decoder =
DecoderRNN(embed_size,
hidden_size, vocab_size,
num_layers)

        def forward(self, images,
captions):
            features =
self.encoder(images)
            return
self.decoder(features,
captions)

        def caption_image(self,
image, vocab, max_len=20):
```

```

        with torch.no_grad():
            features =
self.encoder(image)
            ids =
self.decoder.generate(feature
s, vocab, max_len)
            words =
[vocab.idx2word.get(i,
Vocabulary.UNK) for i in ids]
            words = [w for w in
words if w not in
(Vocabulary.START,
Vocabulary.END)]
            return "
".join(words)

```

```

# -----
-----
-----
# Training
# -----
-----

```

```
-----  
def train(args):  
    device =  
    torch.device("cuda" if  
    torch.cuda.is_available()  
    else "cpu")  
  
    all_captions = []  
    with open(args.captions,  
    newline="", encoding="utf-8")  
    as f:  
        for row in  
    csv.reader(f):  
            if len(row) >= 2:  
  
    all_captions.append(row[1])  
  
    vocab =  
    Vocabulary(freq_threshold=2)  
    vocab.build(all_captions)  
    print(f"Vocabulary size:  
    {len(vocab)}")
```

```
dataset =  
CaptionDataset(args.images,  
args.captions, vocab,  
IMAGE_TRANSFORM)  
loader =  
DataLoader(dataset,  
batch_size=args.batch_size,  
shuffle=True,  
  
collate_fn=collate_fn)  
  
model =  
ImageCaptioningModel(embed_si  
ze=256, hidden_size=512,  
  
vocab_size=len(vocab)).to(dev  
ice)  
criterion =  
nn.CrossEntropyLoss(ignore_in  
dex=0)  
optimizer =  
torch.optim.Adam(
```

```
        [p for p in
model.parameters() if
p.requires_grad], lr=1e-3)

    for epoch in
range(args.epochs):
        model.train()
        total_loss = 0.0
        for images, captions
in loader:
            images, captions
= images.to(device),
captions.to(device)

        optimizer.zero_grad()
            outputs =
model(images, captions)
            loss = criterion(

outputs.reshape(-1,
outputs.size(-1)),
captions.reshape(-1))
            loss.backward()
```

```
            optimizer.step()
            total_loss +=
loss.item()

        print(f"Epoch {epoch
+ 1}/{args.epochs} - loss:
{total_loss /
len(loader):.4f}")

    torch.save({
        "model_state":
model.state_dict(),
        "vocab": vocab,
    }, args.checkpoint)
    print(f"Saved checkpoint
to {args.checkpoint}")

def caption(args):
    device =
torch.device("cuda" if
torch.cuda.is_available()
else "cpu")
```

```
checkpoint =  
torch.load(args.checkpoint,  
map_location=device,  
weights_only=False)  
vocab =  
checkpoint["vocab"]  
  
model =  
ImageCaptioningModel(embed_si  
ze=256, hidden_size=512,  
  
vocab_size=len(vocab)).to(dev  
ice)  
  
model.load_state_dict(checkpo  
int["model_state"])  
model.eval()  
  
image =  
Image.open(args.image).conver  
t("RGB")  
image =
```

```
IMAGE_TRANSFORM(image).unsqueeze(0).to(device)
```

```
    result =  
model.caption_image(image,  
vocab)  
    print(f"Generated  
caption: {result}")
```

```
# -----  
-----  
-----
```

```
# CLI
```

```
# -----  
-----  
-----
```

```
def main():  
    parser =  
argparse.ArgumentParser(description="Image Captioning (CNN  
+ RNN)")  
    subparsers =
```

```
parser.add_subparsers(dest="command", required=True)

    train_p =
subparsers.add_parser("train"
, help="Train the captioning
model")
    train_p.add_argument("--
images", required=True,
help="Path to images folder")
    train_p.add_argument("--
captions", required=True,
help="Path to captions CSV")
    train_p.add_argument("--
epochs", type=int,
default=20)
    train_p.add_argument("--
batch_size", type=int,
default=32)
    train_p.add_argument("--
checkpoint",
default="caption_model.pt")
```

```
train_p.set_defaults(func=train)

    caption_p =
subparsers.add_parser("caption", help="Caption a single
image")
    caption_p.add_argument("-
image", required=True,
help="Path to the image
file")
    caption_p.add_argument("-
checkpoint",
default="caption_model.pt")

caption_p.set_defaults(func=caption)

    args =
parser.parse_args()
    args.func(args)
```

```
if __name__ == "__main__":  
    main()
```